# Software Requirements Specification for

# Gerenciamento de Pedidos de Assistência Técnica

**Version 1.0**

**Prepared by Arthur Vasconcelos de Oliveira e Bruno Alencar**

**Instituto Federal do Sertão Pernambucano - IFSertão-PE**

**09 de novembro de 2026**

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

# Table of Contents

**Revision History** **3**
**1. Introduction** **3**
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Project Scope 4
1.4 References 4
**2. Overall Description** **4**
2.1 Product Perspective 4
2.2 User Classes and Characteristics 5
2.3 Operating Environment 5
2.4 Design and Implementation Constraints 6
2.5 Assumptions and Dependencies 6
**3. System Features** **6**
3.1 Gestão de Clientes 6
3.2 Gestão de Equipamentos 7
3.3 Gestão de Técnicos 7
3.4 Gestão de Ordens de Serviço 8
3.5 Histórico de Status 8
3.6 Documentação da API 9
**4. Data Requirements** **9**
4.1 Logical Data Model 9
4.2 Data Dictionary 10
4.3 Reports 11
4.4 Data Acquisition, Integrity, Retention, and Disposal 11
**5. External Interface Requirements** **11**
5.1 User Interfaces 11
5.2 Software Interfaces 12
5.3 Hardware Interfaces 13
5.4 Communications Interfaces 13
**6. Quality Attributes** **13**
6.1 Usability 13
6.2 Performance 13
6.3 Security 13
6.4 Safety 14
**7. Internationalization and Localization Requirements** **14**
**8. Other Requirements** **14**
**Appendix A: Glossary** **15**
**Appendix B: Analysis Models** **16**
**Appendix C: Use Case Models** **18**

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| Arthur Vasconcelos | 13/10/2026 | Definição do tema e levantamento inicial das entidades (cliente, equipamento, ordem de serviço) | 0.1 |
| Arthur Vasconcelos e Bruno Alencar | 29/10/2026 | Título e escopo fechados; primeiro rascunho dos requisitos funcionais | 0.2 |
| Bruno Alencar | 03/11/2026 | Inclusão das entidades técnico e histórico de status; DER revisado | 0.3 |
| Arthur Vasconcelos | 06/11/2026 | Regras de negócio | 0.4 |
| Bruno Alencar | 07/11/2026 | Diagrama de estados da OS e requisitos não funcionais | 0.5 |
| Arthur Vasconcelos e Bruno Alencar | 09/11/2026 | Revisão geral, endpoints da API, casos de uso e versão para entrega | 1.0 |

# 1. Introduction

## 1.1 Purpose

Este documento especifica os requisitos de software da API REST de Gerenciamento de Pedidos de Assistência Técnica, versão 1.0. A API controla o ciclo de vida das ordens de serviço (OS) de uma assistência técnica de informática, desde a entrada do equipamento até a entrega ao cliente.

O documento destina-se a:

- Desenvolvedores - para orientar a implementação das entidades, regras de negócio e endpoints;
- Avaliadores acadêmicos - para verificação de conformidade com o roteiro do projeto da disciplina;
- Testadores - para elaboração de casos de teste no Postman/Insomnia e testes automatizados com JUnit e Mockito.

## 1.2 Document Conventions

- Requisitos funcionais são identificados com o prefixo RF- seguido de número sequencial (ex.: RF-01).
- Requisitos não funcionais são identificados com o prefixo RNF- (ex.: RNF-01).
- Regras de negócio são identificadas com o prefixo RN- (ex.: RN-01).
- Prioridade dos requisitos: Alta, Média ou Baixa.
- Nomes de campos, rotas e valores de status aparecem em `fonte monoespaçada`.
- Termos técnicos e siglas são definidos no Apêndice A (Glossário).
- O sistema é referenciado como “API de Pedidos” ao longo do documento.

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

## 1.3 Project Scope

A API de Pedidos é o back-end de uma assistência técnica de informática que atende serviços como formatação, instalação de softwares, limpeza física, manutenção e reparos de hardware. Assistências de pequeno porte costumam controlar esses pedidos em cadernos, planilhas ou WhatsApp, o que gera equipamentos sem dono identificado e orçamentos aprovados sem registro. A API integra:

- Cadastro de clientes, equipamentos e técnicos;
- Abertura e acompanhamento de ordens de serviço, com laudo técnico e orçamento;
- Controle de status com transições válidas e histórico de cada mudança;
- Consultas com paginação e filtros por status, cliente e técnico;
- Documentação interativa dos endpoints via Swagger.

Ficam fora desta versão: interface gráfica, autenticação e perfis de acesso, controle de estoque de peças, emissão de nota fiscal e notificações automáticas ao cliente.

## 1.4 References


- Roteiro do projeto “Desenvolvimento de uma API REST com JavaEE, Spring e Hibernate” - disciplina de JEE, IFSertão-PE Campus Salgueiro
- WIEGERS, K.; BEATTY, J. Software Requirements. 3. ed. Redmond: Microsoft Press, 2013 (modelo de SRS)
- Spring Boot Documentation - https://spring.io/projects/spring-boot
- Spring Data JPA Documentation - https://spring.io/projects/spring-data-jpa
- Hibernate ORM Documentation - https://hibernate.org/orm/documentation/
- PostgreSQL Documentation - https://www.postgresql.org/docs/
- springdoc-openapi - https://springdoc.org

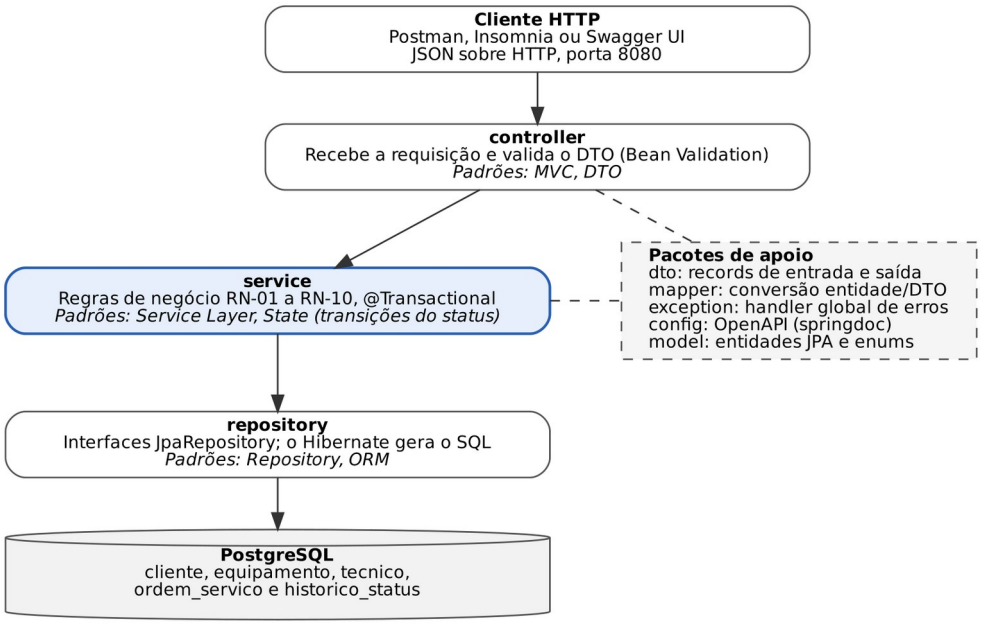

# 2. Overall Description

## 2.1 Product Perspective

A API de Pedidos é um sistema novo e independente, desenvolvido como projeto acadêmico da disciplina de JEE do curso de Tecnologia em Sistemas para Internet do IFSertão-PE. Não é continuação de nenhum produto existente e não substitui sistema anterior.

O sistema é um back-end RESTful construído com Java e Spring Boot, organizado em camadas (controller, service, repository e model). A persistência usa Spring Data JPA com Hibernate sobre PostgreSQL. Qualquer cliente HTTP (Postman, Insomnia, Swagger UI ou um front-end futuro) consome a API trocando JSON.

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*




***Fonte: Pesquisa Direta***

## 2.2 User Classes and Characteristics

**Classe 1 - Atendente / recepção**

- Usuário principal no balcão da assistência.
- Características: conhecimento básico de informática; precisa registrar pedidos com rapidez.
- Acesso: cadastra clientes e equipamentos, abre OS, registra a entrega e informa o status ao cliente.

**Classe 2 - Técnico**

- Responsável pela análise e pelo reparo dos equipamentos.
- Características: conhecimento técnico em hardware e software.
- Acesso: registra laudo técnico, orçamento e serviços executados; altera o status das OS em que trabalha.

**Classe 3 - Gerente / dono**

- Acompanha a fila de serviços e a equipe.
- Acesso adicional: cadastro e desativação de técnicos, atribuição de técnico às OS e consultas filtradas.

**Classe 4 - Cliente final**

- Não acessa a API nesta versão; recebe informações sobre o andamento pelo atendente.

## 2.3 Operating Environment

**Servidor (API de Pedidos):**

- Plataforma: Java 17 (JDK 17 ou superior), Spring Boot 3
- Build: Maven 3.9+

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

- Banco de dados: PostgreSQL 16, local ou em contêiner Docker
- Sistemas operacionais: Windows, Linux ou macOS

**Cliente:**

- Postman ou Insomnia para testes dos endpoints
- Navegador moderno para acessar o Swagger UI

## 2.4 Design and Implementation Constraints

- Linguagem e frameworks: Java com Spring Boot, Spring Data JPA e Hibernate, conforme o roteiro da disciplina.
- Banco de dados: relacional (MySQL ou PostgreSQL); a equipe adotou PostgreSQL.
- Estrutura de pacotes: `controller`, `service`, `repository` e `model`, com `Application.java` na raiz.
- Validação: todos os dados de entrada validados com Bean Validation antes de chegar à camada de serviço.
- Contrato da API: entidades JPA nunca são expostas; a API recebe e devolve apenas DTOs.
- Versionamento: código mantido no GitHub.
- Credenciais do banco lidas de variáveis de ambiente (`DB_URL`, `DB_USER`, `DB_PASSWORD`).

## 2.5 Assumptions and Dependencies

**Premissas:**

- A API roda na rede interna da assistência; por isso a autenticação fica para uma versão futura.
- Cada equipamento pertence a um único cliente.
- O orçamento é aprovado verbalmente pelo cliente e registrado pelo atendente na API.

**Dependências externas:**

- PostgreSQL: disponibilidade essencial para todas as funcionalidades com persistência.
- Ecossistema Spring e bibliotecas do Maven Central para build da aplicação.
- Docker (opcional): para subir o banco via Docker Compose.

# 3. System Features

## 3.1 Gestão de Clientes

**Descrição:** Prioridade Alta. Cadastro das pessoas ou empresas que deixam equipamentos na assistência, com consulta, atualização e remoção.

**Stimulus/Response Sequences:**

- Atendente envia `POST /api/clientes` com dados válidos → sistema valida, persiste e retorna 201 com o cliente criado.
- Atendente tenta cadastrar e-mail ou CPF/CNPJ já existente → sistema retorna 409 com a mensagem “Já existe um cliente com este e-mail.”

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

- Atendente busca por nome parcial → sistema retorna página de clientes cujo nome contém o termo.
- Atendente tenta remover cliente com equipamentos → sistema bloqueia com status 409.

**Functional Requirements:**

| ID | Requisito |
|---|---|
| RF-01 | O sistema deve permitir o cadastro de clientes com nome, telefone, e-mail e CPF/CNPJ opcional. |
| RF-02 | O sistema deve listar clientes com paginação e filtro por nome parcial. |
| RF-03 | O sistema deve permitir consultar, atualizar e remover cliente por ID. |
| RF-04 | O sistema deve rejeitar cadastros com e-mail ou CPF/CNPJ duplicado com status HTTP 409 (RN-06). |

## 3.2 Gestão de Equipamentos

**Descrição:** Prioridade Alta. Registro dos equipamentos trazidos pelos clientes, que são o objeto de cada ordem de serviço.

**Stimulus/Response Sequences:**

- Atendente envia `POST /api/equipamentos` com `clienteId` existente → sistema vincula o equipamento ao cliente e retorna 201.
- Atendente informa `clienteId` inexistente → sistema retorna 404.
- Atendente consulta `GET /api/clientes/{id}/equipamentos` → sistema lista os equipamentos do cliente.
- Atendente tenta remover equipamento que possui OS → sistema bloqueia com status 409.

**Functional Requirements:**

| ID | Requisito |
|---|---|
| RF-05 | O sistema deve permitir o cadastro de equipamento vinculado a um cliente, com tipo, marca, modelo, número de série e especificações. |
| RF-06 | O sistema deve listar os equipamentos de um cliente. |
| RF-07 | O sistema deve permitir consultar, atualizar e remover equipamento por ID. |

## 3.3 Gestão de Técnicos

**Descrição:** Prioridade Média. Manutenção da equipe técnica que analisa e repara os equipamentos.

**Stimulus/Response Sequences:**

- Gerente envia `POST /api/tecnicos` → sistema cria o técnico como ativo e retorna 201.
- Gerente tenta cadastrar e-mail já existente → sistema retorna 409.
- Gerente envia `DELETE /api/tecnicos/{id}` → sistema marca o técnico como inativo e retorna 204, sem apagar o registro.

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

**Functional Requirements:**

| ID | Requisito |
|---|---|
| RF-08 | O sistema deve permitir o cadastro de técnicos com nome, e-mail e especialidade. |
| RF-09 | O sistema deve listar técnicos, com filtro por ativos. |
| RF-10 | O sistema deve permitir atualizar técnico e desativá-lo por exclusão lógica (`ativo = false`). |

## 3.4 Gestão de Ordens de Serviço

**Descrição:** Prioridade Alta. Núcleo do sistema: abertura, acompanhamento e encerramento de cada pedido de assistência, com controle de status.

**Stimulus/Response Sequences:**

- Atendente envia `POST /api/ordens-servico` com equipamento e descrição do defeito → sistema cria a OS em `AGUARDANDO_ANALISE`, preenche a data de abertura e registra a primeira entrada do histórico.
- Técnico envia `PUT /api/ordens-servico/{id}` com laudo, serviços e valor → sistema atualiza a OS e retorna 200.
- Gerente atribui técnico inativo → sistema retorna 422 “Técnico inativo não pode receber OS.”
- Usuário solicita transição não permitida → sistema retorna 422 “Transição de EM_REPARO para ENTREGUE não permitida.”
- Técnico muda a OS para `CONCLUIDO` → sistema valida os serviços descritos e preenche a data de conclusão.
- Gerente lista OS com filtros → sistema retorna página filtrada por status, cliente e técnico.

**Functional Requirements:**

| ID | Requisito |
|---|---|
| RF-11 | O sistema deve permitir abrir OS para um equipamento com descrição do defeito; status inicial `AGUARDANDO_ANALISE` e data de abertura automática. |
| RF-12 | O sistema deve permitir registrar laudo técnico, descrição dos serviços e valor total. |
| RF-13 | O sistema deve permitir atribuir ou trocar o técnico responsável, apenas entre técnicos ativos. |
| RF-14 | O sistema deve alterar o status respeitando as transições válidas (Apêndice B). |
| RF-15 | O sistema deve preencher a data de conclusão e a data de entrega ao entrar em `CONCLUIDO` e `ENTREGUE`. |
| RF-16 | O sistema deve permitir cancelar OS que ainda não entrou em reparo. |
| RF-17 | O sistema deve listar OS com paginação e filtros por status, cliente e técnico. |

## 3.5 Histórico de Status

**Descrição:** Prioridade Média. Rastreabilidade de cada mudança de status das ordens de serviço.

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

**Stimulus/Response Sequences:**

- Qualquer mudança de status → sistema grava status anterior, status novo, data/hora e observação.
- Usuário consulta `GET /api/ordens-servico/{id}/historico` → sistema retorna as mudanças em ordem cronológica.

**Functional Requirements:**

| ID | Requisito |
|---|---|
| RF-18 | O sistema deve registrar automaticamente cada mudança de status com status anterior, novo, data/hora e observação. |
| RF-19 | O sistema deve permitir consultar o histórico de uma OS em ordem cronológica. |

## 3.6 Documentação da API

**Descrição:** Prioridade Baixa. Documentação interativa gerada a partir dos controllers, para testes e integração.

**Functional Requirements:**

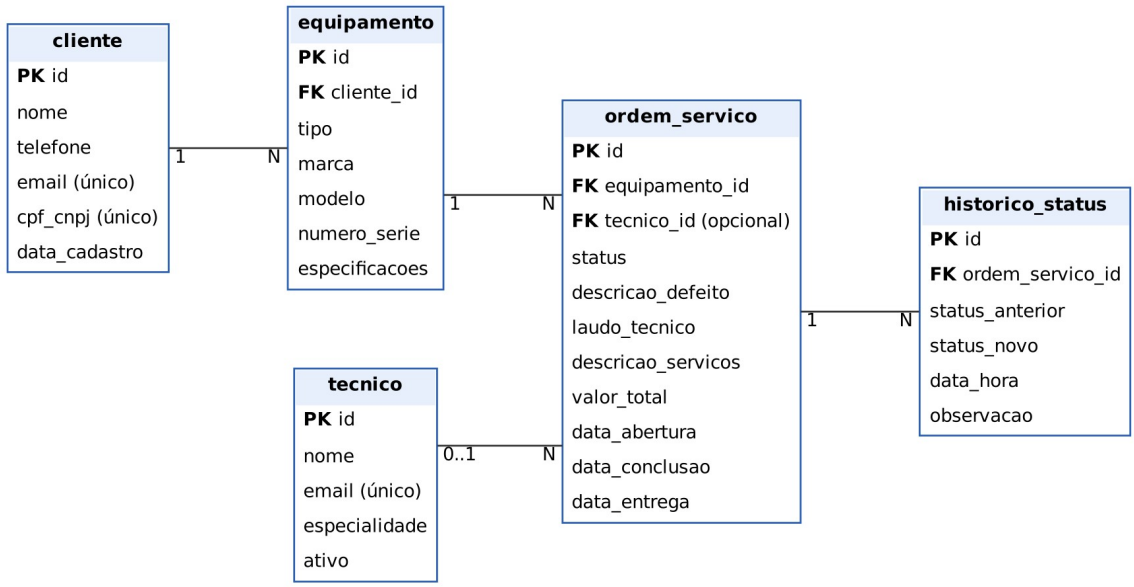
| ID | Requisito |
|---|---|
| RF-20 | O sistema deve disponibilizar documentação interativa em `/swagger-ui.html` e a especificação OpenAPI em `/v3/api-docs`. |

# 4. Data Requirements

## 4.1 Logical Data Model

O modelo tem cinco tabelas: um cliente possui vários equipamentos, cada equipamento pode ter várias OS ao longo do tempo, cada OS tem no máximo um técnico e várias entradas de histórico.

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*




***Fonte: Pesquisa Direta***

## 4.2 Data Dictionary

| Entidade | Campo | Tipo | Descrição |
|---|---|---|---|
| cliente | id | BIGINT | PK, auto-incremento |
| cliente | nome | VARCHAR(120) | Nome ou razão social; obrigatório |
| cliente | telefone | VARCHAR(20) | Telefone principal; obrigatório |
| cliente | email | VARCHAR(120) | E-mail de contato; único |
| cliente | cpf_cnpj | VARCHAR(18) | Documento do cliente; único, opcional |
| cliente | data_cadastro | TIMESTAMP | Preenchida na criação |
| equipamento | id | BIGINT | PK, auto-incremento |
| equipamento | cliente_id | BIGINT | FK para cliente.id; obrigatório |
| equipamento | tipo | VARCHAR(20) | 'NOTEBOOK', 'DESKTOP', 'IMPRESSORA', 'MONITOR', 'OUTRO' |
| equipamento | marca | VARCHAR(60) | Fabricante; obrigatório |
| equipamento | modelo | VARCHAR(80) | Modelo comercial; obrigatório |
| equipamento | numero_serie | VARCHAR(80) | Número de série; opcional |
| equipamento | especificacoes | VARCHAR(500) | Configuração e acessórios entregues |
| tecnico | id | BIGINT | PK, auto-incremento |
| tecnico | nome | VARCHAR(120) | Nome do técnico; obrigatório |
| tecnico | email | VARCHAR(120) | E-mail; único, obrigatório |
| tecnico | especialidade | VARCHAR(80) | Ex.: hardware, software, impressoras |
| tecnico | ativo | BOOLEAN | Exclusão lógica; padrão true |
| ordem_servico | id | BIGINT | PK; número da OS |
| ordem_servico | equipamento_id | BIGINT | FK para equipamento.id; obrigatório |

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

| Entidade | Campo | Tipo | Descrição |
|---|---|---|---|
| ordem_servico | tecnico_id | BIGINT | FK para tecnico.id; opcional |
| ordem_servico | status | VARCHAR(25) | Ver diagrama de estados (Apêndice B) |
| ordem_servico | descricao_defeito | VARCHAR(1000) | Relato do cliente; obrigatório |
| ordem_servico | laudo_tecnico | VARCHAR(2000) | Diagnóstico do técnico |
| ordem_servico | descricao_servicos | VARCHAR(2000) | Serviços executados |
| ordem_servico | valor_total | NUMERIC(10,2) | Valor do orçamento; não negativo |
| ordem_servico | data_abertura | TIMESTAMP | Preenchida na abertura |
| ordem_servico | data_conclusao | TIMESTAMP | Preenchida em `CONCLUIDO` |
| ordem_servico | data_entrega | TIMESTAMP | Preenchida em `ENTREGUE` |
| historico_status | id | BIGINT | PK, auto-incremento |
| historico_status | ordem_servico_id | BIGINT | FK para ordem_servico.id; obrigatório |
| historico_status | status_anterior | VARCHAR(25) | Status de origem; nulo na abertura |
| historico_status | status_novo | VARCHAR(25) | Status de destino; obrigatório |
| historico_status | data_hora | TIMESTAMP | Momento da mudança |
| historico_status | observacao | VARCHAR(500) | Comentário do usuário |

### 4.3 Reports

O sistema não possui relatórios formais em formato de documento. Os seguintes indicadores estão disponíveis pelas consultas da API:

- Fila de serviços: listagem de OS filtrada por status, útil para ver o que está aguardando análise, em reparo ou pronto para entrega.
- Carga por técnico: listagem de OS filtrada por técnico responsável.
- Histórico do pedido: sequência de mudanças de status de uma OS, com data/hora e observação.

### 4.4 Data Acquisition, Integrity, Retention, and Disposal

- Integridade referencial: garantida por chaves estrangeiras entre cliente, equipamento, técnico, ordem de serviço e histórico.
- Unicidade: e-mail e CPF/CNPJ de cliente e e-mail de técnico reforçados por constraint UNIQUE; a violação é tratada pela API com status 409.
- Retenção de OS: nenhuma ordem de serviço é apagada fisicamente; o encerramento ocorre por `ENTREGUE` ou `CANCELADO`, e os dados são mantidos para histórico.
- Remoções: clientes e equipamentos só podem ser removidos quando não possuem registros dependentes (RN-07); técnicos são desativados em vez de excluídos.

## 5. External Interface Requirements

### 5.1 User Interfaces

A versão 1.0 não possui interface gráfica própria. A interação humana com a API ocorre pelas seguintes ferramentas:

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

| Interface | Uso |
|---|---|
| Swagger UI (`/swagger-ui.html`) | Consulta da documentação e execução dos endpoints pelo navegador |
| Postman / Insomnia | Testes manuais com a coleção de requisições versionada no repositório |

## 5.2 Software Interfaces

**PostgreSQL:**

- Acesso: Spring Data JPA com Hibernate como implementação de JPA e driver JDBC oficial do PostgreSQL.
- Operações: CRUD via interfaces `JpaRepository` e consultas derivadas dos nomes dos métodos.

**springdoc-openapi:**

- Geração automática da especificação OpenAPI 3 a partir dos controllers.

**Endpoints REST:**

| Método | Rota | Descrição | RF |
|---|---|---|---|
| POST | `/api/clientes` | Cadastra cliente | RF-01 |
| GET | `/api/clientes?nome=` | Lista clientes | RF-02 |
| GET | `/api/clientes/{id}` | Consulta cliente | RF-03 |
| PUT | `/api/clientes/{id}` | Atualiza cliente | RF-03 |
| DELETE | `/api/clientes/{id}` | Remove cliente | RF-03 |
| GET | `/api/clientes/{id}/equipamentos` | Equipamentos do cliente | RF-06 |
| POST | `/api/equipamentos` | Cadastra equipamento | RF-05 |
| GET | `/api/equipamentos/{id}` | Consulta equipamento | RF-07 |
| PUT | `/api/equipamentos/{id}` | Atualiza equipamento | RF-07 |
| DELETE | `/api/equipamentos/{id}` | Remove equipamento | RF-07 |
| POST | `/api/tecnicos` | Cadastra técnico | RF-08 |
| GET | `/api/tecnicos?ativos=` | Lista técnicos | RF-09 |
| GET | `/api/tecnicos/{id}` | Consulta técnico | RF-09 |
| PUT | `/api/tecnicos/{id}` | Atualiza técnico | RF-10 |
| DELETE | `/api/tecnicos/{id}` | Desativa técnico | RF-10 |
| POST | `/api/ordens-servico` | Abre OS | RF-11 |
| GET | `/api/ordens-servico?status=&clienteId=&tecnicoId=` | Lista OS com filtros | RF-17 |
| GET | `/api/ordens-servico/{id}` | Consulta OS | RF-17 |
| PUT | `/api/ordens-servico/{id}` | Registra laudo, serviços e valor | RF-12 |
| PATCH | `/api/ordens-servico/{id}/tecnico` | Atribui técnico | RF-13 |
| PATCH | `/api/ordens-servico/{id}/status` | Muda status | RF-14 a RF-16 |

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

| Método | Rota | Descrição | RF |
|---|---|---|---|
| GET | `/api/ordens-servico/{id}/historico` | Histórico de status | RF-19 |

## 5.3 Hardware Interfaces

Não existem interfaces de hardware específicas. O sistema opera inteiramente via rede, com conexão HTTP padrão.

## 5.4 Communications Interfaces

- Protocolo HTTP/1.1: toda comunicação com a API via REST, na porta 8080 por padrão.
- Formato de dados: exclusivamente JSON em UTF-8 para requisições e respostas.
- Paginação: listagens aceitam os parâmetros `page`, `size` e `sort`.
- Formato de erro: toda falha devolve `timestamp`, `status`, `erro`, `mensagem`, `caminho` e, em erro de validação, `campos`.
- Códigos HTTP: 200, 201, 204, 400 (validação), 404 (não encontrado), 409 (duplicidade ou dependência) e 422 (regra de negócio).
- JDBC: conexão com o PostgreSQL na porta 5432.

# 6. Quality Attributes

## 6.1 Usability

| ID | Requisito |
|---|---|
| RNF-01 | Mensagens de erro devem ser claras e em português, indicando o campo inválido e o motivo (ex.: “O campo telefone é obrigatório.” em vez de “Error 400”). |
| RNF-02 | As rotas devem seguir convenções REST: recursos no plural, verbos HTTP adequados e nomes de campos em camelCase. |

## 6.2 Performance

| ID | Requisito |
|---|---|
| RNF-03 | Listagens devem ser paginadas, com 20 itens por página como padrão. |
| RNF-04 | Consultas por ID e listagens paginadas devem responder em menos de 500 ms em ambiente local. |

## 6.3 Security

| ID | Requisito |
|---|---|
| RNF-05 | As credenciais do banco devem ser configuradas via variáveis de ambiente, nunca fixas no código-fonte versionado. |
| RNF-06 | Todos os dados de entrada devem ser validados com Bean Validation antes de qualquer processamento. |

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

| ID | Requisito |
|---|---|
| RNF-07 | Entidades JPA não devem ser expostas nas respostas; a API devolve apenas DTOs com os campos necessários. |

### 6.4 Safety

| ID | Requisito |
|---|---|
| RNF-08 | Operações de escrita devem ser transacionais (`@Transactional`), sem gravação parcial em caso de erro. |
| RNF-09 | A integridade referencial deve ser garantida por chaves estrangeiras no banco. |
| RNF-10 | Ordens de serviço nunca devem ser apagadas fisicamente, preservando o histórico de atendimento. |

## 7. Internationalization and Localization Requirements

| ID | Requisito |
|---|---|
| RNF-11 | A linguagem padrão e exclusiva das mensagens e da documentação é o Português do Brasil (pt-BR). |
| RNF-12 | Datas devem trafegar em ISO 8601 (ex.: `2026-11-09T14:30:00`) e valores monetários em reais (BRL), com ponto decimal no JSON. |
| RNF-13 | O fuso horário padrão da aplicação deve ser `America/Recife`. |

A internacionalização para outros idiomas é considerada fora do escopo da versão 1.0.

## 8. Other Requirements

| ID | Requisito |
|---|---|
| RNF-14 | A API deve ser sem estado (stateless), permitindo várias instâncias atrás de um balanceador. |
| RNF-15 | O código deve seguir a arquitetura em camadas, com DTOs separando o contrato da API das entidades JPA. |
| RNF-16 | As regras de negócio devem ter testes unitários com JUnit 5 e Mockito; o fluxo principal deve ter teste de integração com H2 em memória. |
| RNF-17 | O banco deve poder ser iniciado via Docker Compose, permitindo rodar o projeto sem instalar o PostgreSQL. |
| RNF-18 | O código-fonte deve ser versionado no GitHub, com README contendo pré-requisitos, comandos de execução e lista de endpoints. |
| RNF-19 | A API deve registrar erros não tratados no console do servidor para apoio ao diagnóstico. |

**Regras de negócio:**

| ID | Requisito |
|---|---|
| RN-01 | Toda OS nasce com status `AGUARDANDO_ANALISE` e gera a primeira entrada no histórico. |

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

| ID | Requisito |
|---|---|
| RN-02 | Só são aceitas as transições do diagrama de estados (Apêndice B); qualquer outra retorna 422. |
| RN-03 | Para ir a `ORCAMENTO_APROVADO`, a OS precisa ter laudo técnico e valor total preenchidos. |
| RN-04 | Para ir a `EM_REPARO`, a OS precisa ter um técnico ativo atribuído. |
| RN-05 | Para ir a `CONCLUIDO`, a OS precisa ter a descrição dos serviços preenchida. |
| RN-06 | E-mail e CPF/CNPJ de cliente e e-mail de técnico são únicos. |
| RN-07 | Cliente com equipamentos e equipamento com OS não podem ser removidos (409). |
| RN-08 | OS em `ENTREGUE` ou `CANCELADO` não aceita mais alterações. |
| RN-09 | Técnico inativo não pode receber OS. |
| RN-10 | O valor total da OS não pode ser negativo. |

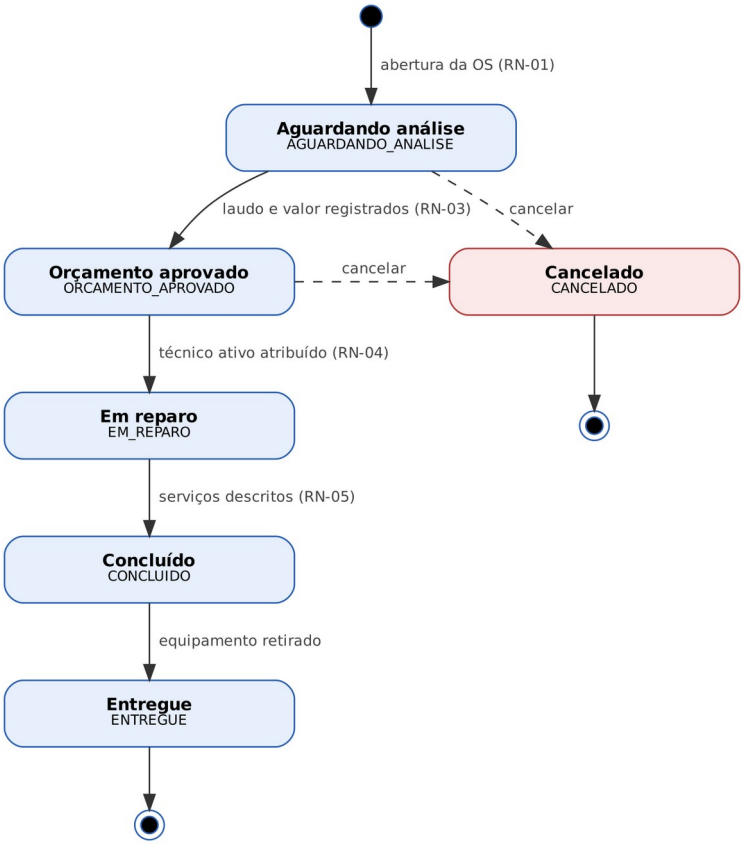
# Appendix A: Glossary

| Termo | Definição |
|---|---|
| API | Application Programming Interface - conjunto de rotas que expõe as funcionalidades do sistema para outros programas |
| REST | Representational State Transfer - estilo arquitetural para APIs baseadas em HTTP |
| OS | Ordem de serviço - registro de um pedido de assistência para um equipamento |
| Laudo técnico | Diagnóstico do defeito feito pelo técnico, base do orçamento |
| DER | Diagrama Entidade-Relacionamento |
| DTO | Data Transfer Object - objeto que transporta dados entre a API e as camadas internas |
| JPA | Jakarta Persistence API - especificação Java de persistência objeto-relacional |
| Hibernate | Framework ORM que implementa a especificação JPA |
| ORM | Object-Relational Mapping - mapeamento entre objetos Java e tabelas do banco |
| Spring Boot | Framework que simplifica a configuração e execução de aplicações Spring |
| Bean Validation | Especificação Java para validação de dados por anotações |
| PostgreSQL | Sistema gerenciador de banco de dados relacional de código aberto |
| Swagger / OpenAPI | Padrão e ferramenta de documentação interativa de APIs |
| Postman / Insomnia | Ferramentas para envio e teste de requisições HTTP |
| JUnit / Mockito | Bibliotecas Java para testes unitários e criação de objetos simulados |

***Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.***

# Appendix B: Analysis Models

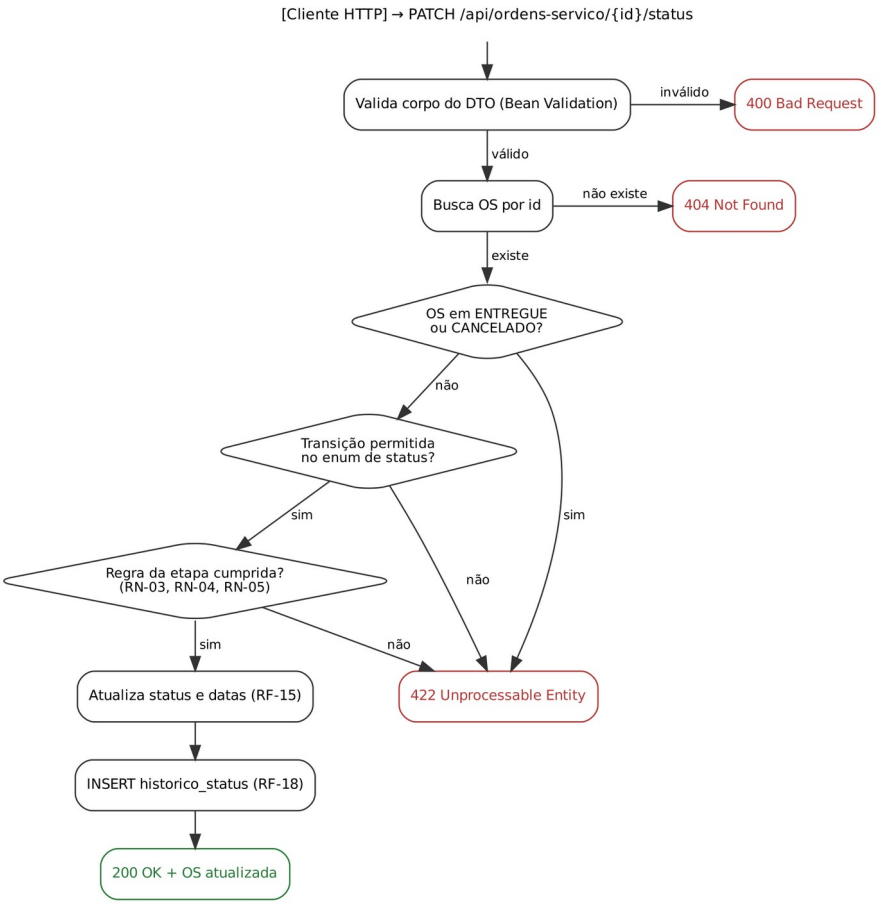
## Diagrama de Estados - Ordem de Serviço




***Fonte: Pesquisa Direta***

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

## Diagrama de Fluxo - Alteração de Status da OS

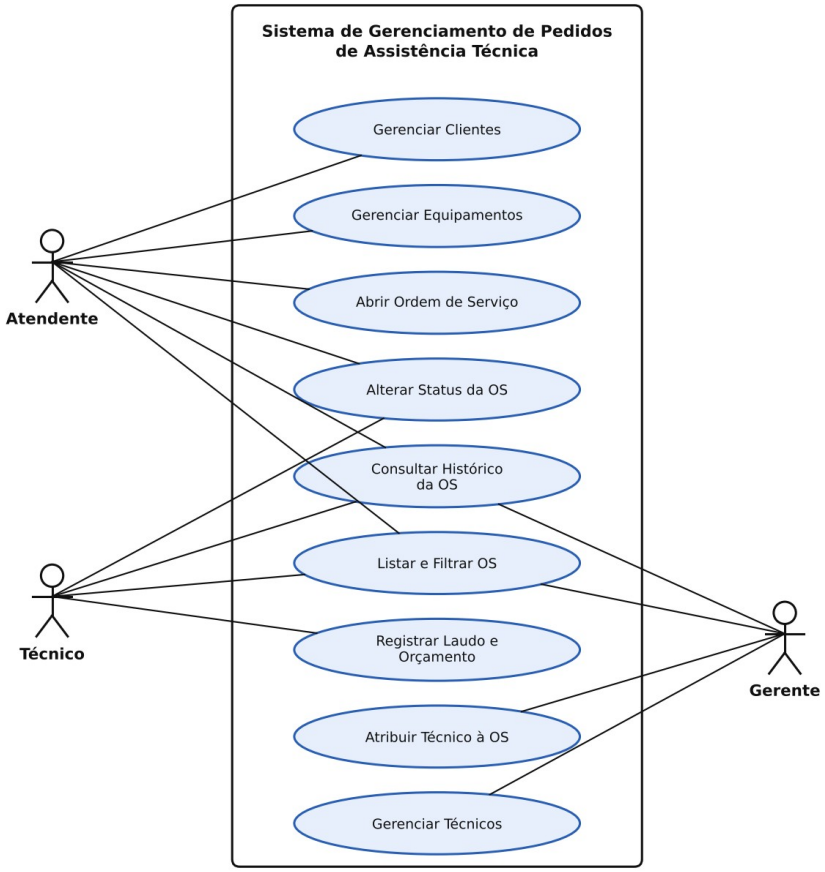



***Fonte: Pesquisa Direta***

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

# Appendix C: Use Case Models

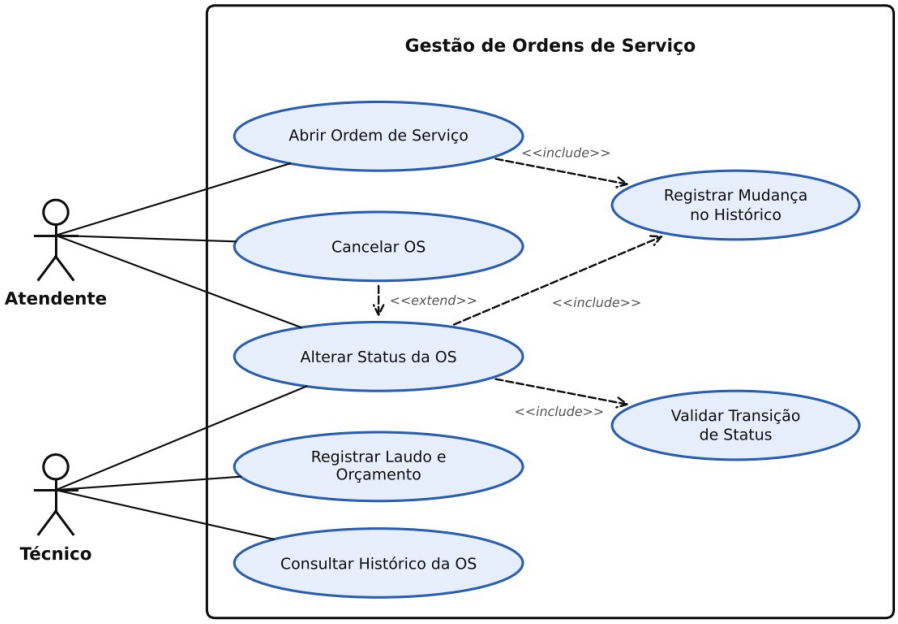
## Diagrama de Caso de Uso - Visão Geral do Sistema




***Fonte: Pesquisa Direta***

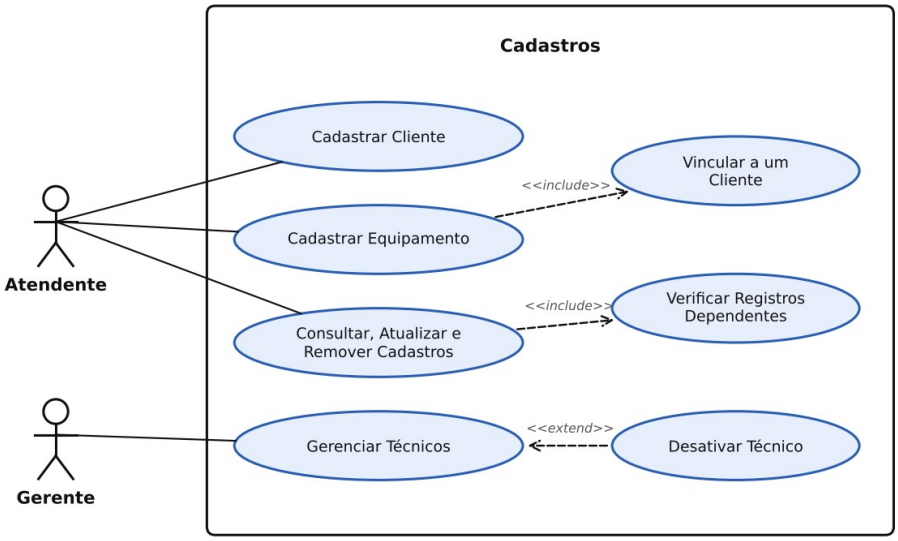
*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*

## Diagrama de Caso de Uso - Gestão de Ordens de Serviço




***Fonte: Pesquisa Direta***

## Diagrama de Caso de Uso - Cadastros




***Fonte: Pesquisa Direta***

*Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.*